

Building a Text Preprocessing Pipeline for NLP

Text preprocessing is the process of converting unstructured text into a clean and consistent representation. Machine-learning algorithms cannot work directly with paragraphs, HTML, spelling variations, or unnecessary symbols, so preprocessing is usually the first stage of an NLP workflow.

A good pipeline should be repeatable: the same transformations must be applied to training data, test data, and new text received after deployment.

1. Install the tools

This example uses Python and NLTK:

```
pip install nltk
```

Download the WordNet data used by the lemmatizer once:

```
import nltk

nltk.download("wordnet")
nltk.download("omw-1.4")
```

The download commands are normally run during environment setup, not every time the application processes text.

2. Decide what to preserve

Preprocessing is not simply a list of operations to apply blindly. The correct steps depend on the task:

- Lowercasing is useful when capitalization does not carry meaning.
- Punctuation may be important for sentiment, questions, or programming-language text.
- Numbers may be important for finance, dates, or medical records.
- Stop words can be removed for some classification tasks, but removing them can damage meaning in phrases such as **not good**.
- Stemming is fast but can create words that are not valid dictionary entries.
- Lemmatization is more meaningful but usually requires more linguistic information.

3. Clean the raw text

The following function performs common cleaning operations while keeping the steps explicit:

```

import html
import re

def clean_text(text):
    if not isinstance(text, str):
        raise TypeError("text must be a string")

    text = html.unescape(text)
    text = re.sub(r"<[^>]*>", " ", text)      # Remove HTML tags
    text = re.sub(r"https?://\S+|www\.\S+", " ", text)  # Remove URLs
    text = text.lower()
    text = re.sub(r"[^a-z\s]", " ", text)      # Keep letters and spaces
    return re.sub(r"\s+", " ", text).strip()

```

For example, the input:

```
<p>Learning NLP is fun!</p> Visit https://example.com
```

can become:

```
learning nlp is fun visit
```

The regular expressions shown above are suitable for simple English text. Multilingual data, emojis, hashtags, and domain-specific symbols require a more careful character policy.

4. Tokenize the text

Tokenization divides text into meaningful units called tokens. For simple whitespace-separated English text:

```

cleaned_text = clean_text(raw_text)
tokens = cleaned_text.split()

```

For more complex text, use a tokenizer from an NLP library because contractions, punctuation, and special symbols may need separate handling.

5. Remove stop words selectively

Stop words are common words such as **the**, **is**, and **and**. A small, task-specific set is often safer than automatically removing every word from a default list:

```
STOP_WORDS = {"a", "an", "and", "are", "as", "the", "is", "of", "to", "in"}

filtered_tokens = [
    token for token in tokens
    if token not in STOP_WORDS
]
```

Always check the effect of stop-word removal on the task. For sentiment analysis, retaining negation words such as **not** is especially important.

6. Apply stemming or lemmatization

Stemming uses simple rules to shorten words:

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()
stems = [stemmer.stem(token) for token in filtered_tokens]
```

For example, related words may be reduced to a common form such as **connect**, although the result might not be a complete word.

Lemmatization aims to produce a valid dictionary form:

```
from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()
lemmas = [lemmatizer.lemmatize(token) for token in filtered_tokens]
```

Lemmatization is generally easier to interpret. Its quality improves when the correct part of speech is supplied:

```
lemma = lemmatizer.lemmatize("running", pos="v")
print(lemma) # run
```

Choose either stemming or lemmatization based on the model and evaluation results; using both is not always necessary.

7. Combine the stages

A reusable pipeline can return each intermediate representation for inspection:

```
def preprocess_text(text):
    cleaned_text = clean_text(text)
    tokens = cleaned_text.split()
    filtered_tokens = [
        token for token in tokens
        if token not in STOP_WORDS
    ]

    return {
        "cleaned_text": cleaned_text,
        "tokens": filtered_tokens,
        "stems": [stemmer.stem(token) for token in filtered_tokens],
        "lemmas": [lemmatizer.lemmatize(token) for token in filtered_tokens],
    }
```

Example usage:

```
result = preprocess_text("The students are learning NLP.")
print(result["tokens"])
print(result["lemmas"])
```

Returning a dictionary makes it possible to compare the output of different preprocessing choices instead of losing the original intermediate results.

8. Validate the pipeline

Test normal input as well as edge cases:

```
assert clean_text(" NLP!! ") == "nlp"
assert preprocess_text("") ["tokens"] == []

try:
    preprocess_text(None)
except TypeError:
    pass
else:
    raise AssertionError("Non-string input should raise TypeError")
```

Also inspect a few real examples manually. A pipeline can run without errors and still remove information that the NLP task needs.

9. Common mistakes

1. Applying different preprocessing to training and test data.

2. Removing punctuation or numbers that carry domain meaning.
3. Removing negation words in sentiment or intent classification.
4. Assuming stemming always creates valid English words.
5. Downloading external NLP resources inside every prediction request.
6. Cleaning text before preserving information needed for feature extraction.

Conclusion

A general preprocessing workflow is:

```
raw text -> validation -> HTML/URL cleanup -> normalization  
-> tokenization -> optional stop-word filtering  
-> stemming or lemmatization -> model features
```

The best pipeline is the smallest one that improves the target task without removing useful information. Start with simple, measurable transformations and compare alternatives using validation data.